Software Development and Applications

Variant 1

Final Report

Name: Subrahmanya Hara Shashank Mattigunta

Matriculation Number: 100004907

Course: Software Development and Applications (k_ESTM_009)

Overview

This report presents my solutions to the five programming tasks of Variant 1. Rather than treating the questions in isolation, I approached the set as a small portfolio: each task is a standalone script (Q1.py–Q5.py) that runs under Python 3.11+ from a dedicated virtual environment and ends with a demonstration block. Throughout the report I first state the idea behind my solution, then show the exact console output produced on my machine, and finally reflect on anything I had to be careful about.

Only the Python standard library is used, except in Task 5 where the `openai` client library talks to a locally hosted language model through LM Studio, as permitted by the exam rules. The code was submitted as a GitHub repository to satisfy the repository bonus.

Task	Focus area	Result
Q1	Lists & string parsing	Runs, all outputs correct
Q2	Dictionaries & set logic	Runs, set-based solution
Q3	Variadic functions	Runs, four demo cases
Q4	Classes & inheritance	Runs, all methods exercised
Q5	LLM JSON pipeline + bonus	Runs, bonus completed

Task 1 — Movie Ratings

Idea

The input is one long string of "title:rating" pairs. I read it into a dictionary that maps each film to a list of its scores, splitting first on commas and then on the colon, and applying `.strip()` so that stray spaces around a title or a number never reach the integer conversion. Keeping every score in a list (rather than a running total) means the same parsed structure serves all four queries — average, best film, and counts — without re-reading the raw text.

Console Output

```
Q1: MOVIE RATINGS
Parsed movies: {'Dune': [8, 9, 10], 'Barbie': [7, 9, 6],
                'Oppenheimer': [9]}
Dune average: 9.0
Barbie average: 7.3
Oppenheimer average: 9.0
Unknown movie: 0.0
Best movie: Dune
Rating counts: {'Dune': 3, 'Barbie': 3, 'Oppenheimer': 1}
Extra spaces test: {'Dune': [8, 10], 'Barbie': [9]}
```

Reflection

The care point was type handling: a rating read from text is a string, so it must be converted to int after stripping whitespace, otherwise averaging fails. The final "extra spaces test" line confirms the parser is whitespace tolerant. Dune and Oppenheimer share an average of 9.0; my best-film query returns the first film reaching the top average, which is acceptable since the task does not prescribe tie-breaking.

Task 2 — Gradebook

Idea

Grades are held per student as a mapping of subject to grade. The instruction that matters most is that "students taking every subject" must come from set operations, not counting. I therefore build a set of students for each subject and intersect them, so only students appearing in all subjects survive. Averages ignore missing students by returning 0.0 when the grade list is empty, and the honour roll is produced by filtering on the average and sorting alphabetically.

Console Output

```
Q2: GRADEBOOK
Student data: {'anna': {'math': 1.7, 'art': 1.0},
  'ben': {'math': 2.3, 'physics': 3.0},
  'clara': {'math': 1.0, 'physics': 1.3},
  'david': {'physics': 2.0, 'art': 1.7}}
anna subjects: {'art', 'math'}      anna average: 1.35
ben subjects: {'physics', 'math'}   ben average: 2.65
clara subjects: {'physics', 'math'} clara average: 1.15
david subjects: {'art', 'physics'}  david average: 1.85
Students taking all subjects: set()
Unknown student: 0.0
Honor roll: ['anna', 'clara']
```

Reflection

The empty set for "all subjects" is correct: no student here is enrolled in every subject, and set intersection expresses that far more cleanly than a counting loop. The main defensive check is avoiding a division-by-zero for a student who appears nowhere, handled by returning 0.0. The honour roll contains anna (1.35) and clara (1.15), the only averages at or below the 1.5 threshold.

Task 3 — Recipe Scaler

Idea

This task is about a flexible signature: `scale_recipe(name, servings, *ingredients, unit="g", **options)`. The starred parameter absorbs any number of (ingredient, amount) pairs, `unit` sits after the star so it is keyword-only, and `**options` collects free-form cooking notes. Each amount is multiplied by the serving count and printed with the unit; a serving count below one prints an error and returns 0, otherwise a dictionary of scaled amounts is returned.

Console Output

```
Q3: RECIPE SCALER

Pasta   Servings: 4
  pasta: 400 g
  tomato sauce: 800 g
  cheese: 200 g

Smoothie   Servings: 2      (unit = ml)
  milk: 500 ml
  yogurt: 200 ml

Cake   Servings: 6
  flour: 1200 g
  sugar: 900 g
  Notes:  oven: 180C   time: 45min

ERROR: Pizza needs at least 1 serving
```

Reflection

Placing `unit` after `*ingredients` is the subtlety otherwise it would be swallowed as another ingredient pair. The four demonstrations cover the required range: a plain recipe, a unit override to millilitres, a recipe carrying keyword notes, and the guard case with zero servings.

Task 4 — Library System

Idea

A `Book` holds title, author and year, with `__str__` for display, `__eq__` defining equality by title and author, and an `age()` helper. `EBook` extends it with a file size, a richer string, and a download-time method. The `Library` keeps a list and leans on `__eq__` inside `add()` so a duplicate is silently ignored; it also offers author search, an oldest-book query, and `__len__`.

Console Output

```
Q4: LIBRARY SYSTEM
Number of books: 5
Orwell: ['1984 by George Orwell (1949)',
         'Animal Farm by George Orwell (1945)']
Oldest: The Great Gatsby by F. Scott Fitzgerald (1925)
Age of 1984: 77
Dune download time: 2.0 seconds
EBook example: Dune by Frank Herbert (1965) [2.5 MB]
```

Reflection

The download calculation needs a unit conversion: a size in megabytes becomes megabits by multiplying by eight before dividing by a link speed in Mbit/s, so a 2.5 MB file at 10 Mbit/s takes 2.0 seconds. The book count staying at five confirms the duplicate add was rejected through `__eq__`, exactly as intended.

Task 5 — Movie Review Extraction Pipeline

Idea

This task drives a small local model (`qwen3-0.6b` via LM Studio at `localhost:1234`) to turn free-text reviews into a strict JSON object, and guards against the model misbehaving. The chain is: send a prompt at temperature 0, parse the reply, validate it, and — if validation fails — send the error back to the model and retry up to three times. Validation is strict: the three keys must be exactly right, the rating must be an integer in 1–10 (a boolean is rejected), and the sentiment must be one of two words.

For my own run I used five reviews written in different styles, including one phrased as a plain sentence and one clearly negative review.

Console Output

```
Title                Rating   Sentiment
-----------------------------------------
Dune                 9        positive
Barbie               6        positive
Liger                3        negative
Akhanda              8        positive
Tenet                7        positive

Positive: 4  Negative: 1
```

Bonus — Temperature Comparison

I ran the same five reviews at two temperatures and recorded how many retries and failures each run needed:

Temperature	Retries	Failures
0.0	0	0
1.5	1	0

Observation: at temperature 0 every review produced valid JSON on the first attempt, but at temperature 1.5 one review needed a retry before it parsed correctly. This is direct evidence from my own run that raising the temperature reduces the consistency of a small model's structured output: the extra randomness occasionally pushes the reply away from the required format, and the retry step is what recovers it. No review failed outright, so the three-attempt guard was sufficient in every case.

Reflection

The difficulties here were mostly about the environment installing the client library into the project's virtual environment and making sure the LM Studio server was actually running and reachable before the script could connect. Once the local server was serving the model, the pipeline ran cleanly, and the single retry at temperature 1.5 turned out to be the most interesting result, since it shows the defensive retry logic doing real work.

Closing Notes

All five scripts run without errors under Python 3.11+ and demonstrate the required behaviour in their own output. Together they cover the module's core ideas: parsing and aggregating text data, reasoning with dictionaries and sets, writing functions with variable and keyword arguments, modelling a small class hierarchy, and wrapping a language model in a validated, self-correcting pipeline. The Task 5 bonus was completed, and the only non-standard dependency used anywhere is the permitted client library in Task 5.

The complete source for all five tasks is in the submitted GitHub repository, one file per question, each with its demonstration block at the foot of the file.